

Copias,

acuerdo y orquestación.

LA PREGUNTA

¿Cómo guardo el saldo en tres máquinas sin que diga dos cosas distintas?



EN UNA FRASE Hoy todo vive en una sola máquina; al terminar, en tres que se ponen de acuerdo solas.

Cuatro paradas,

una decisión.

CUATRO PARADAS

Copiar los datos, qué tan iguales se ven las copias, cómo se ponen de acuerdo y **quién usa ese acuerdo para mantener vivo tu sistema.**

AVISOS

Reporte de v05.1 vence el **2026-09-27**

Hoy no hay lab hay demo del banco partido

Después del descanso una corrección de Raft

Replicación

copias con un líder

Consistencia y CAP

qué tan iguales

Consenso

ponerse de acuerdo

Kubernetes

quién lo usa

La decisión

réplicas o máquina grande

Demo

Mexi Banco partido

EN UNA FRASE Cada parada resuelve un problema que **la anterior deja abierto.**

Tres razones, una copia.

REPLICACIÓN

Tener **una copia de los mismos datos en varias máquinas** conectadas por red.

PARA QUÉ

Latencia los datos cerca del usuario

Disponibilidad sigue aunque falle una máquina

Lecturas el estado de cuenta se lee **mucho más** de lo que se escribe



EN UNA FRASE

Si los datos nunca cambiaran, replicar sería copiar una vez: **todo lo difícil está en los cambios.**

Réplica no es respaldo.

RESPALDO

Una foto vieja para volver en el tiempo. La réplica, en cambio, **copia también el error.**

TRES FAMILIAS

Un líder uno escribe, los demás copian

Multilíder varios escriben

Sin líder el cliente escribe a varios



COMO SI FUERA

Un espejo copia lo que haces al instante; **la foto del álbum se queda como estaba.**

EN UNA FRASE **Las réplicas te protegen de una máquina que falla; los respaldos, de una persona que se equivoca.**

Uno escribe, todos copian.

UN LÍDER

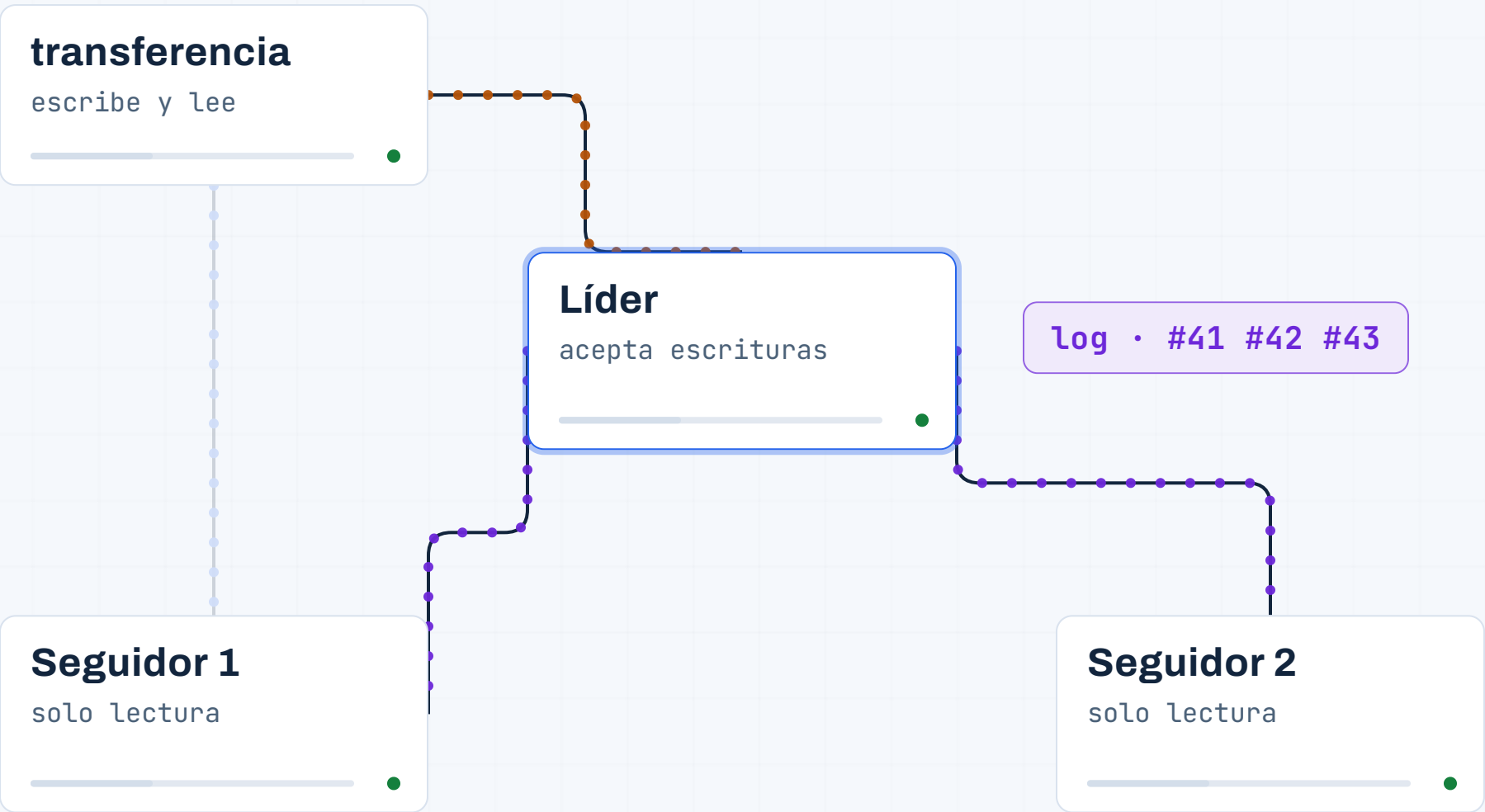
Todas las escrituras van a un nodo, el líder, y los seguidores aplican su log en el mismo orden.

LO QUE SE DESPRENDE

Lecturas en el líder o en cualquier seguidor

Dónde está Postgres, MySQL, Kafka, etcd

Nombre viejo "master-slave"; hoy se dice líder y seguidor



COMO SI FUERA

El libro de movimientos de la matriz: las sucursales no inventan asientos, **copian los de la matriz en el mismo orden.**

Esperar

o no esperar.

SÍNCRONA

El líder **espera la confirmación** del seguidor antes de decir OK; en la asíncrona, no espera.

LO QUE SE DESPRENDE

Todos **síncronos** uno caído detiene todas las escrituras

Mayoría síncrona 3 de 5: regresa después del descanso

El que se cae vuelve y pide todo desde su posición

EN UNA FRASE

Esperar compra que el dato no se pierda; **no esperar compra que el líder no se detenga.**



Muere

el líder. ¿Y ahora?

FAILOVER

Promover a un seguidor a líder y que todos le escriban a él.

TRES PASOS

- 1. **Detectar** con un timeout: el dilema de la S03
- 2. **Elegir** al seguidor **más al día**
- 3. **Reconfigurar** clientes al nuevo; el viejo, a obedecer

EN UNA FRASE

Los tres pasos suenan simples; cada uno esconde una forma de equivocarse.



Lo que sale mal.

SPLIT BRAIN

Dos nodos creen ser líder y los dos aceptan escrituras.

TRES FALLAS

Escrituras perdidas confirmadas al cliente y descartadas

Dos líderes datos corruptos o perdidos

El timeout corto o largo, **ninguno es correcto**

Cliente A

retira 800

Líder viejo

pausado · lo creen muerto

Cliente B

retira 900

Líder nuevo

saldo 1 000

Replicación



COMO SI FUERA

Dos profesores que creen tener el mismo salón a la misma hora: **cada uno pasa lista y los dos califican.**

EN UNA FRASE

Cambiar de líder sin perder datos ni tener dos líderes tiene nombre: consenso. Lo vemos después del descanso.

Tarde o temprano.

CONSISTENCIA EVENTUAL

Si dejas de escribir, **las réplicas terminan iguales**; no dice cuándo.

TRES GARANTÍAS

Leer tus escrituras tu cambio, de una fuente al día
Lecturas monótonas siempre de la misma réplica
Prefijo consistente lo causal, en el mismo orden

EN UNA FRASE

Fingir que la réplica es síncrona cuando es asíncrona **es receta para problemas.**



Varios escriben.

MULTILÍDER Y SIN LÍDER

Varios nodos aceptan escrituras; se paga con conflictos o con cuórums.

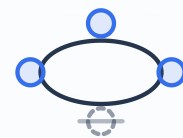
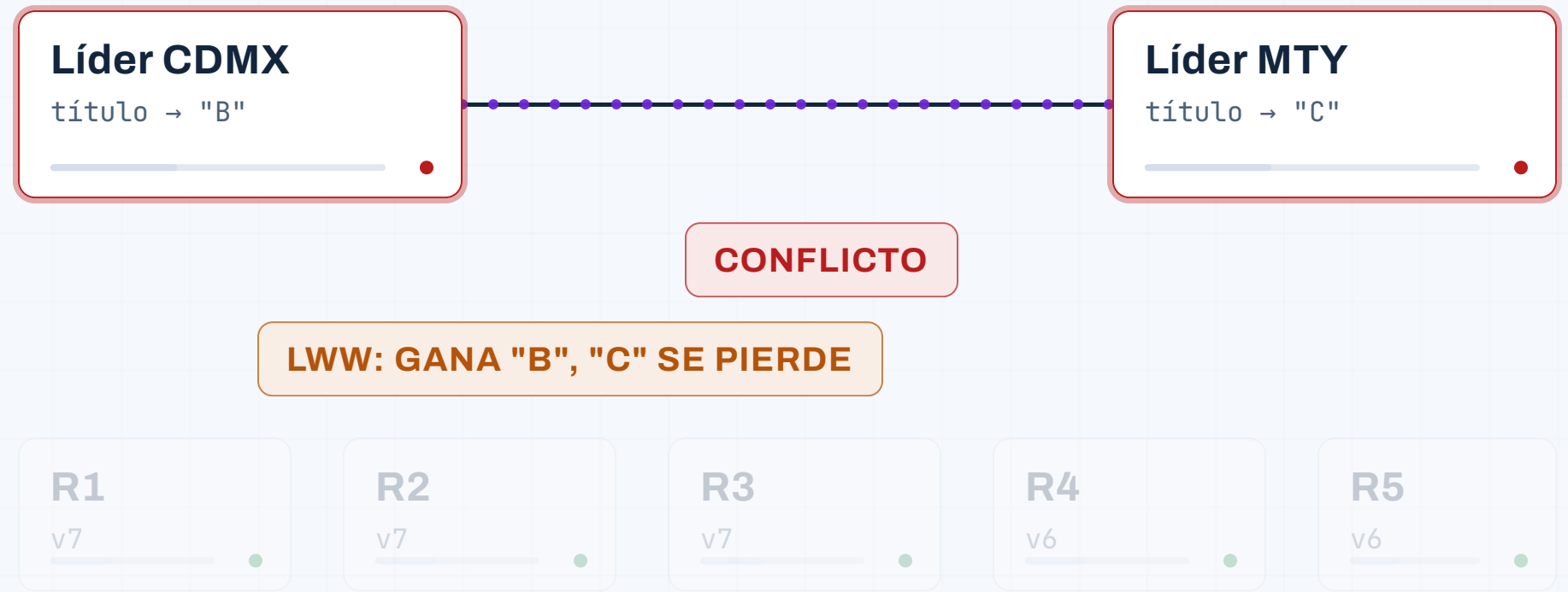
PANORAMA

Multilíder regiones, Git, Docs; **no garantiza saldo ≥ 0**

Sin líder $w + r > n$: escribes a 3, lees de 3, de 5

Ojo Dynamo no es DynamoDB

Replicación



COMO SI FUERA

Dos comités de 3 personas en un salón de 5: **siempre comparten a alguien**, y ese alguien sabe lo último.

EN UNA FRASE

Sin un solo líder el sistema aguanta más, pero **el saldo ya no se puede proteger igual.**

Tres familias,

una tabla.

LA FAMILIA

Escoger cómo replicas es **escoger qué sacrificas** cuando algo falla.

PARA MEXI BANCO

- El saldo un líder
- Notificaciones y preferencias pueden ser multilíder
- Historial de accesos puede ser sin líder

Un líder

Postgres, MySQL, etcd

Multilíder

Docs, Git, geo

Sin líder

Cassandra, Riak

Consistencia

Fuerte

si lees del líder

Débil

con conflictos

Eventual

ajustable con w y r

Si cae un nodo

Failover

el problema difícil

Los demás siguen

Nada

se usa a los que contestan

Si se parte la red

La mitad sin líder

no escribe

Cada lado sigue

y luego resuelve

Sigue

mientras junte w y r

¿Qué garantía

faltó?

COMPROBACIÓN 1

Transfieres 500. La app dice "listo".
Recargas y **tu saldo no cambió**; un
minuto después, sí. ¿Qué garantía
faltó?

VOTA EN EL CHAT

Una letra, sin explicar todavía
Después, en pares si hay empate

A · Lecturas monótonas

no ver retroceder el tiempo

C · Prefijo consistente

lo causał, en orden

B · Leer tus escrituras

ver lo que tú escribiste

D · Durabilidad

que no se pierda

LA LECTURA FUE A UN SEGUIDOR ASÍNCRONO

EN UNA FRASE

Comprométete con una letra antes de discutir: **predecir primero es lo que hace que se quede.**

Como si

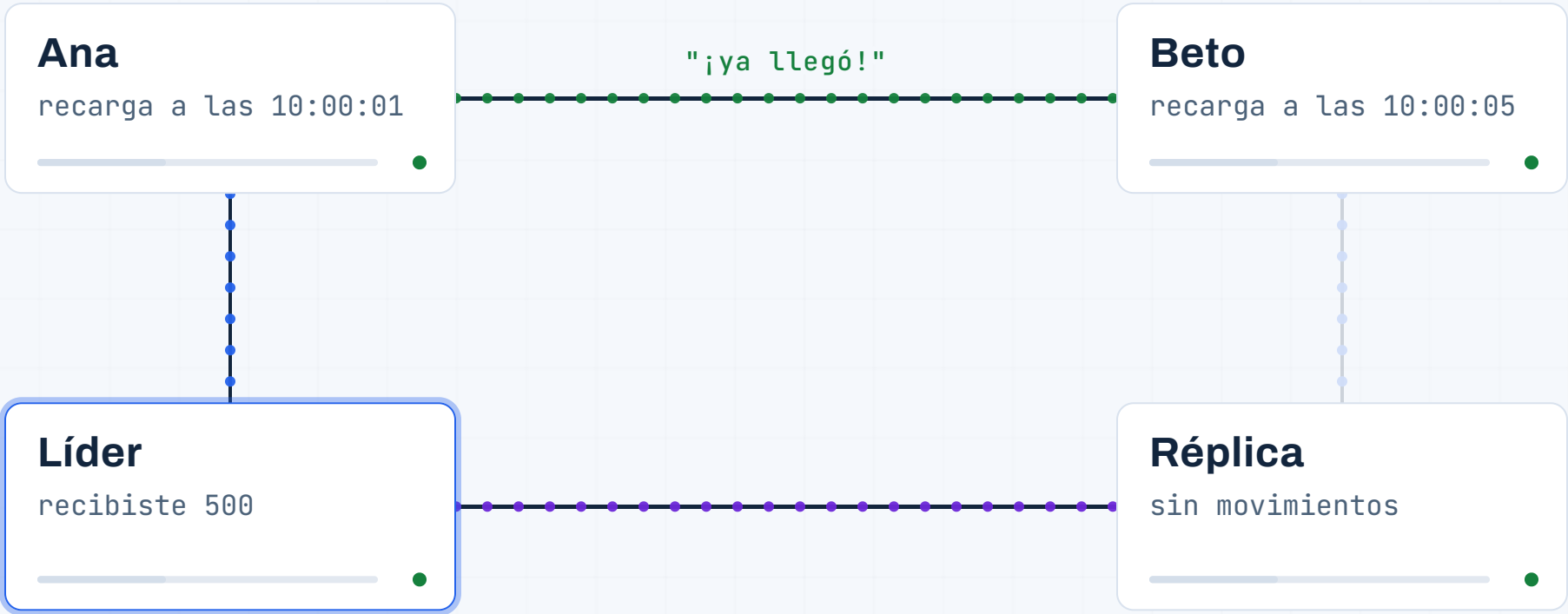
hubiera una sola copia.

LINEALIZABILIDAD

En cuanto alguien ve el valor nuevo, **nadie que empiece después ve el viejo.**

DOS FILOSOFÍAS

- Eventual** la aplicación se encarga
- Fuerte** se comporta como un solo nodo; más lenta
- Origen** Herlihy y Wing, 1990



COMO SI FUERA

El marcador del estadio: todos ven el mismo, y **cuando cambia, cambia para todos a la vez.**

EN UNA FRASE **Linealizable es una garantía de recencia:** lo que ya se vio no puede "desverse".

Cuándo

hace falta, y quién la da.

RESTRICCIÓN DURA

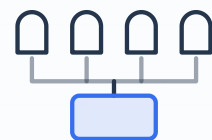
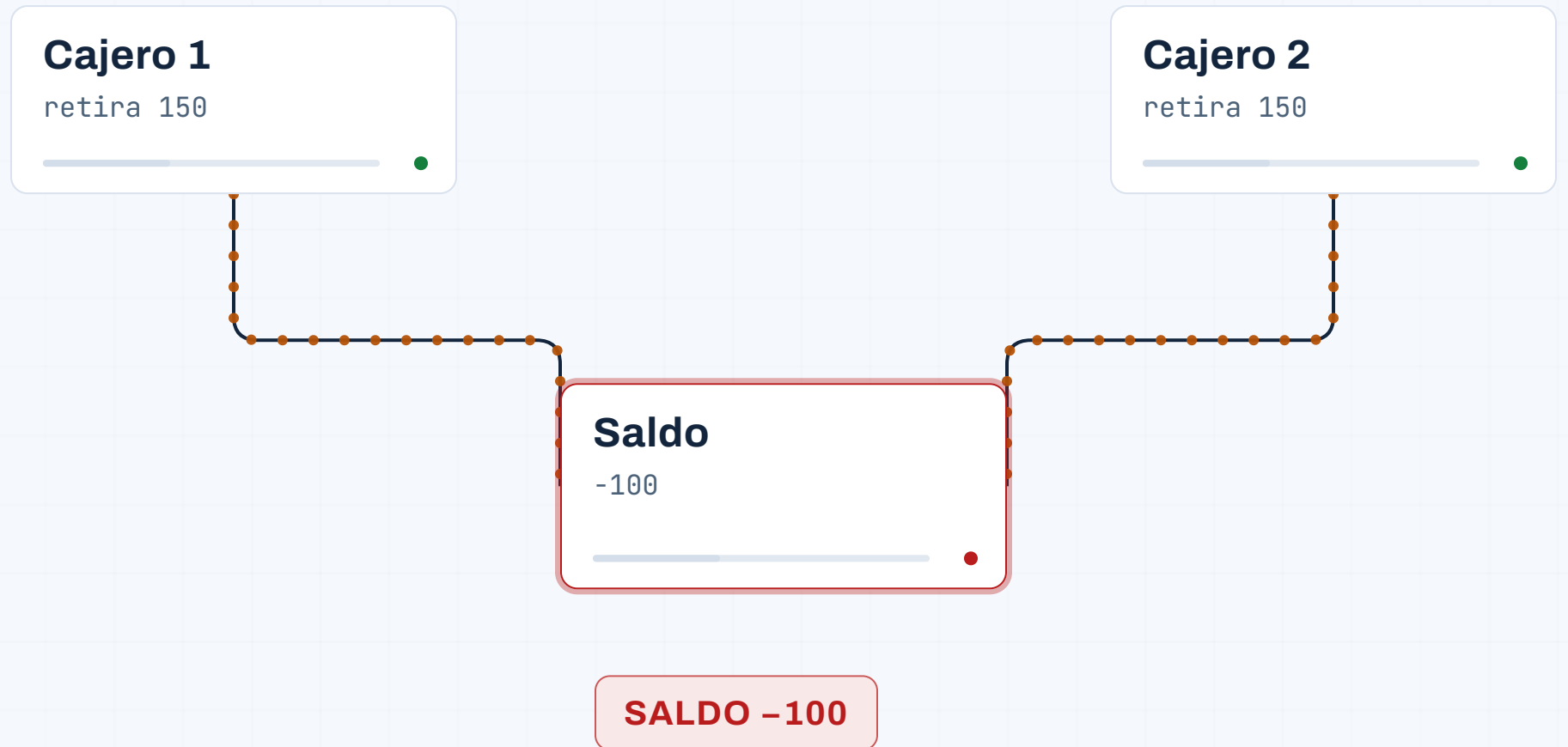
Una regla que necesita **un valor actual en el que todos coinciden.**

LO QUE SE DESPRENDE

Cuándo elegir líder, locks, CLABE única, saldo ≥ 0

La dan un líder leyendo del líder, y el consenso

No la dan multilíder; sin líder casi nunca



COMO SI FUERA

Dos ventanillas que venden el último boleto: **solo una puede decir que sí**, y las dos tienen que saberlo.

EN UNA FRASE

Para el saldo necesitas un valor actual; para las notificaciones, no: se escoge por operación.

La pregunta

de la semana.

PREGUNTA DE LA SEMANA

¿Por qué Brewer dice que "escoge 2 de 3" engaña?

CÓMO VA

Tres respuestas en voz o en el chat
Sin corregir las retomamos en la 16

CAP Twelve Years Later: How the "Rules" Have Changed

Eric Brewer · IEEE Computer · 2012 · Lectura 5

Respuesta 1

la anoto

Respuesta 2

la anoto

Respuesta 3

la anoto

EN UNA FRASE

La lectura no se evalúa por el resumen: se evalúa por la pregunta que te deja.

Se corta el cable.

CAP

Con la red partida, **o respondes con datos al día o respondes en los dos lados.**

LAS DOS SALIDAS

CP el lado sin líder espera o da error

AP los dos lados siguen y reconcilian después

Origen Brewer 2000; prueba de Gilbert y Lynch, 2002



COMO SI FUERA

Dos sucursales sin teléfono: **o dejan de vender, o venden y cuadran la caja después.**

"2 de 3"

engaña.

PARTICIÓN

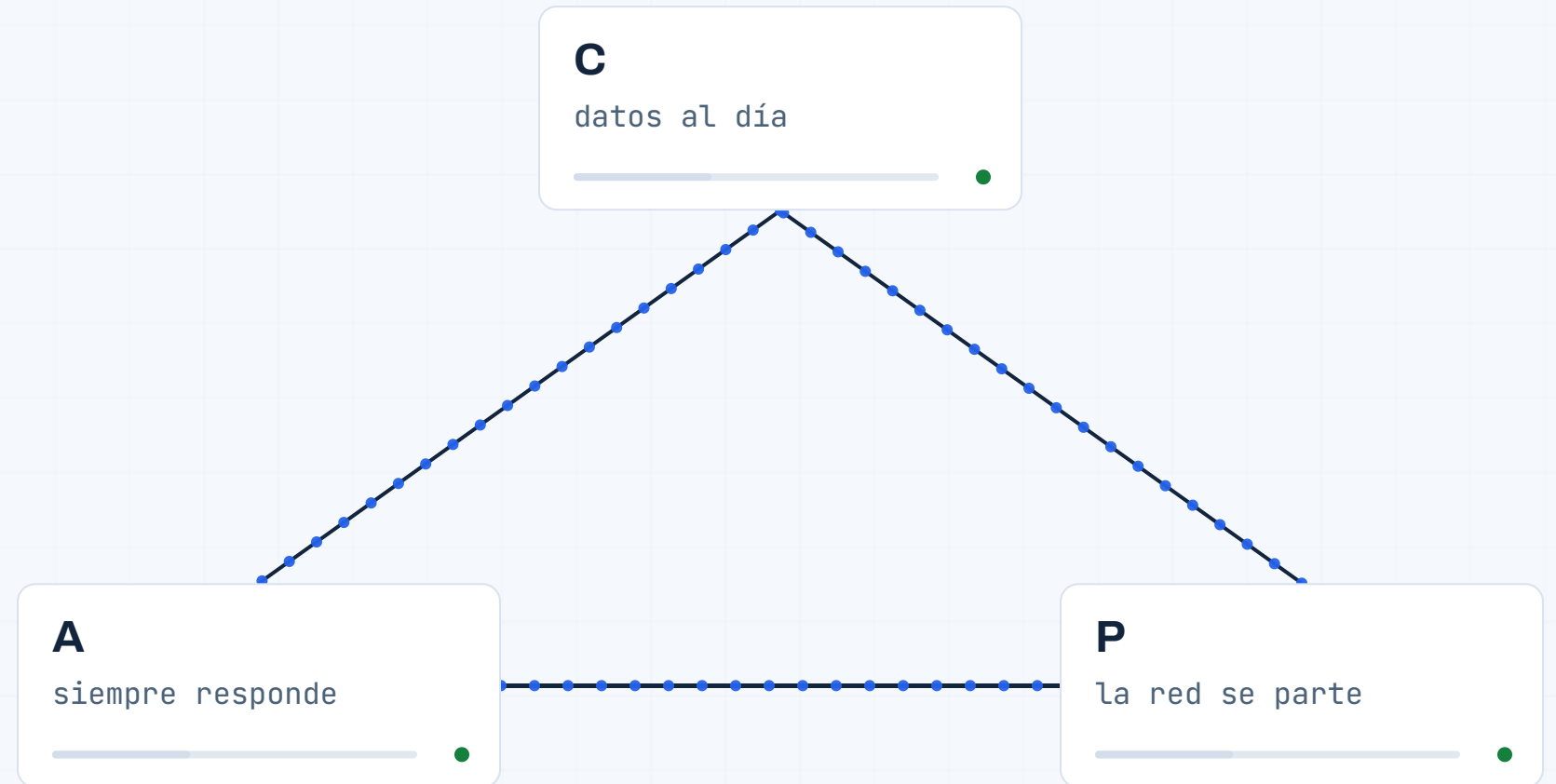
La partición **no se escoge**: llega sola, quieras o no.

LO QUE CORRIGE

Sin partición tienes C y A a la vez

Nada es binario hay grados de C, de A y de P

Es estrecho solo linealizable y solo particiones



¿ESCOGES 2 DE 3?



COMO SI FUERA

Preguntar si quieres llover: **no se escoge**; lo que escoges es si sales con paraguas.

EN UNA FRASE

La frase correcta es **"consistente o disponible, cuando hay partición"**.

17 / PACELC

Y cuando no hay partición.

PACELC

Si hay Partición, A o C; si no (Else),
Latencia o Consistencia.

LO QUE AGREGA

Siempre lo linealizable es lento, no solo en
partición

Ejemplo la RAM de tu laptop no es linealizable

Origen Daniel Abadi, 2012

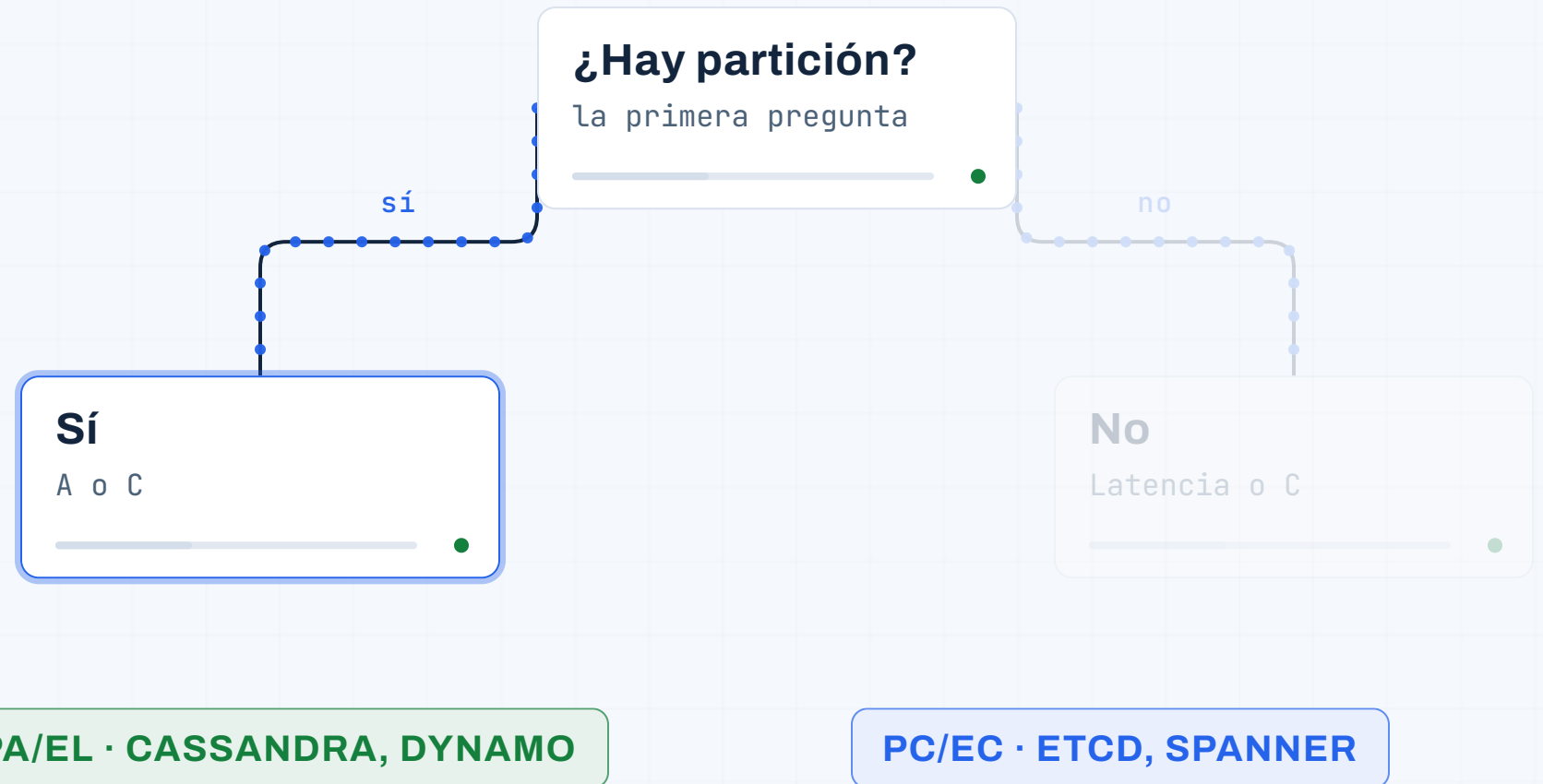


COMO SI FUERA

Confirmar cada venta con la matriz por teléfono: aunque la línea funcione, **cada venta tarda la llamada.**

EN UNA FRASE

La partición es rara; la latencia la pagas en cada petición.



La partición

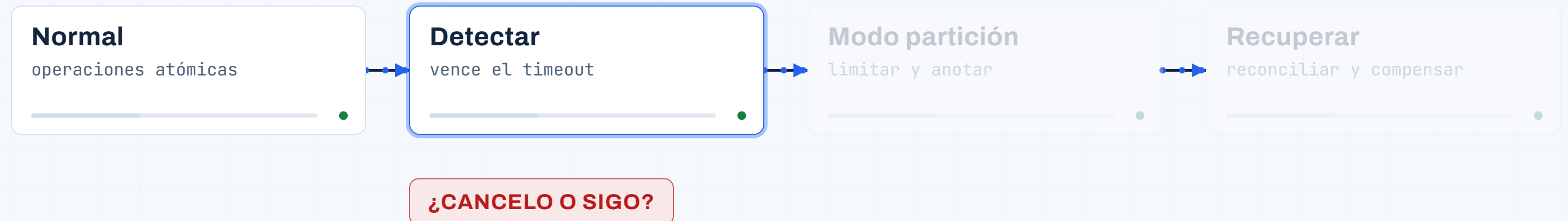
es un timeout.

MODO PARTICIÓN

Detectar, operar limitado, recuperar y compensar: manejar la partición en vez de sufrirla.

BREWER 2012

Al vencer el timeout cancelar pierde A, seguir arriesga C
Reintentar solo pospone la decisión
No hay partición global un lado puede notarla y el otro no



EN UNA FRASE No se sacrifica C o A a ciegas: **se decide por operación, según qué regla no puede romperse.**

El cajero

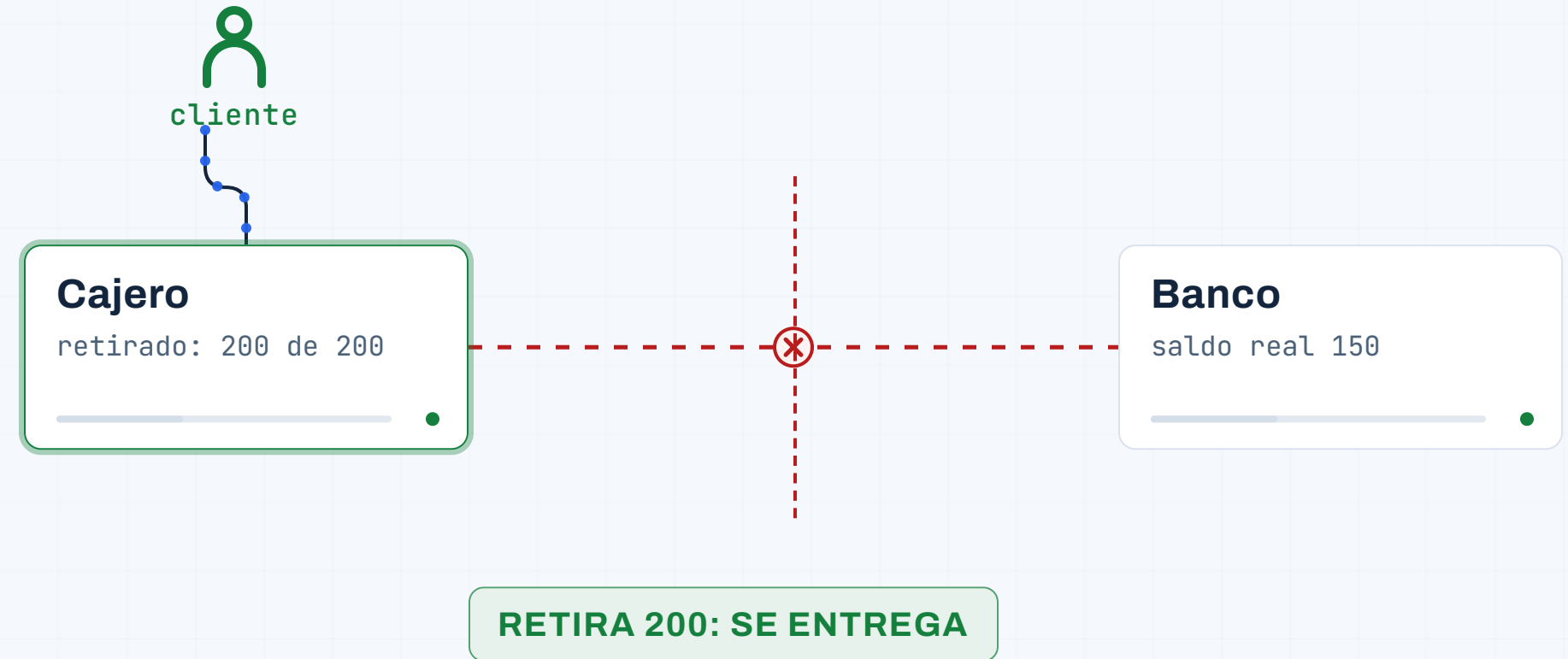
que sigue dando.

MODO STAND-IN

El cajero sin conexión permite **retiros hasta un límite k**; al volver, el banco suma y compensa.

LA REGLA

Invariante saldo ≥ 0 ; solo retirar la rompe
Riesgo acotado nunca más de k
Si quedó negativo comisión y cobro



COMO SI FUERA

La tiendita que te fía hasta 200 pesos porque te conoce: **arriesga poco y cobra después.**

EN UNA FRASE Los bancos no dependen de la consistencia para ser correctos, **sino de la auditoría y la compensación.**

¿Qué dice de verdad CAP?

COMPROBACIÓN 2

Elige la frase que dice **lo que realmente afirma** el teorema CAP.

VOTA EN EL CHAT

Una letra, sin explicar todavía
Después, dos defensas en voz

A · Siempre hay que escoger dos de tres: C, A o P

B · Con la red partida: datos al día o respuesta en los dos lados

C · Sin partición no se puede tener C y A a la vez

D · Con una red suficientemente buena, P se sacrifica

EN UNA FRASE Copiar da disponibilidad; que las copias digan lo mismo cuesta, y **cuando se corta la red hay que escoger.**

21 / DESCANSO

30 minutos.

Vuelve el líder.

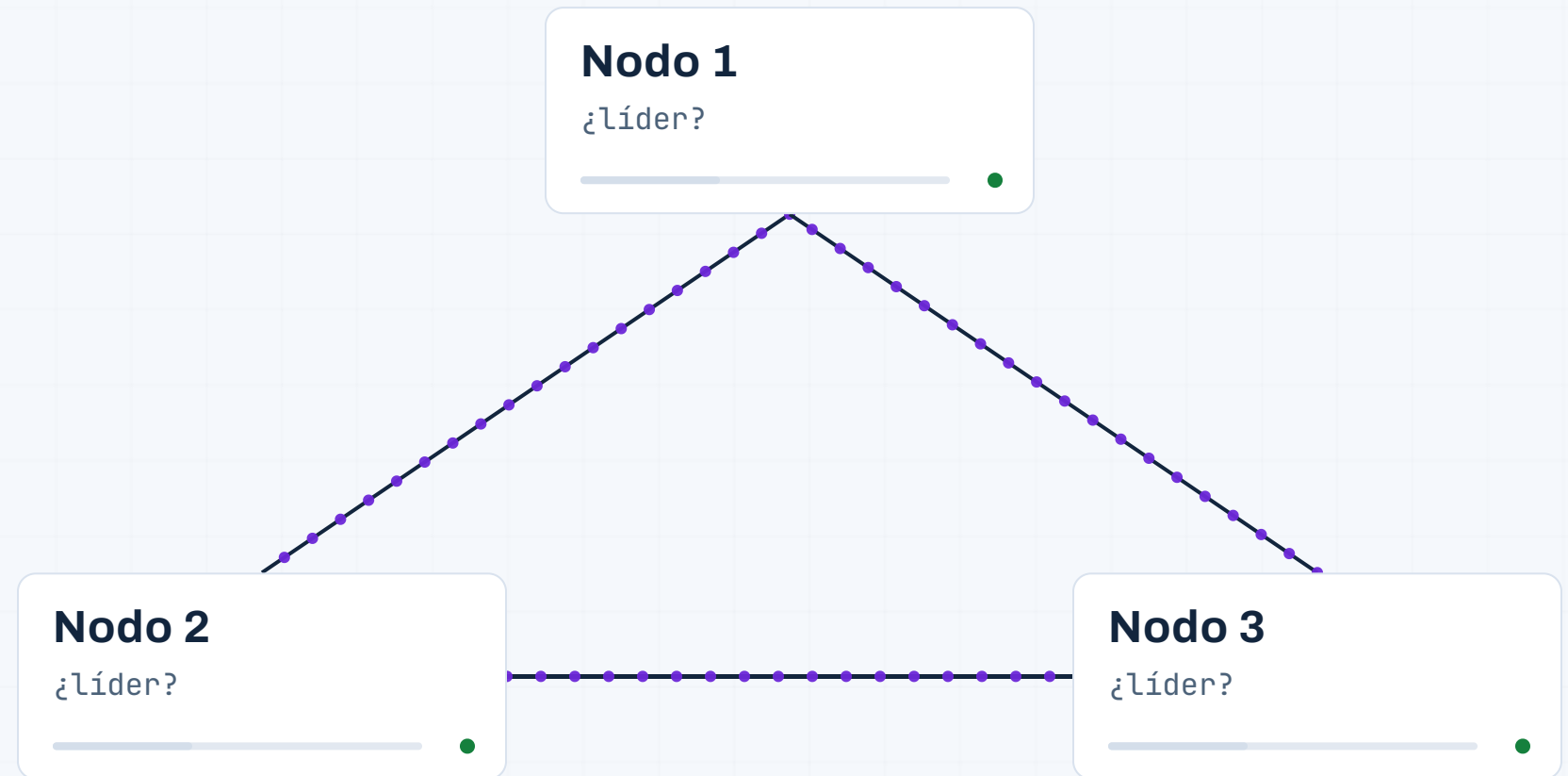
AL VOLVER

¿Cómo se ponen de acuerdo tres máquinas sin un humano?

PENDIENTE

La mayoría 3 de 5 de la lámina 05

El failover seguro de la lámina 07



AL REGRESAR

Primero, una corrección de lo que dije la semana pasada sobre Raft.

El failover, bien hecho.

CONSENSO

Varios nodos **proponen valores y deciden uno**, el mismo para todos; decidir no es promediar.

LA VARIABLE QUE SE ACUERDA

Lock o lease quién lo tiene

CLABE única quién se queda con ella

Log compartido qué va en cada posición

Commit atómico la transacción va o **se deshace**

Lock o lease

la variable es quién lo tiene

CLABE única

la variable es de quién es

Log compartido

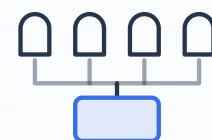
la variable es el orden

Commit atómico

la variable es si va o no va

Consenso

proponer y decidir un valor



COMO SI FUERA

Cuatro ventanillas con cuatro letreros distintos y, al llegar, **el mismo formato en las cuatro**: cambia el trámite, no el procedimiento.

Cuatro cosas que cumplir.

PROPIEDADES

Las **cuatro condiciones** que cualquier algoritmo de consenso tiene que cumplir para ser correcto.

DOS CLASES DE PROPIEDAD

Seguridad acuerdo, integridad y validez: **nunca** deciden mal

Vivacidad terminación: pide una **mayoría viva**

Un **apagón** detiene al grupo, **no lo corrompe**

Terminación

sin ella: el grupo se queda esperando para siempre

Acuerdo

dos que deciden, deciden lo mismo

Integridad uniforme

cada nodo decide a lo más una vez

Validez uniforme

el valor decidido lo propuso alguien

SEGURIDAD: SIEMPRE · VIVACIDAD: SOLO CON MAYORÍA VIVA

CUATRO CASILLAS · SE TACHAN O NO SE TACHAN



COMO SI FUERA

La rúbrica con la que calificas un proyecto: no preguntas si el algoritmo te cae bien, revisas las **cuatro casillas** y marcas la que no cumple.

EN UNA FRASE

Si cae la mayoría, el sistema se detiene; lo que ya decidió no se corrompe.

Un log compartido.

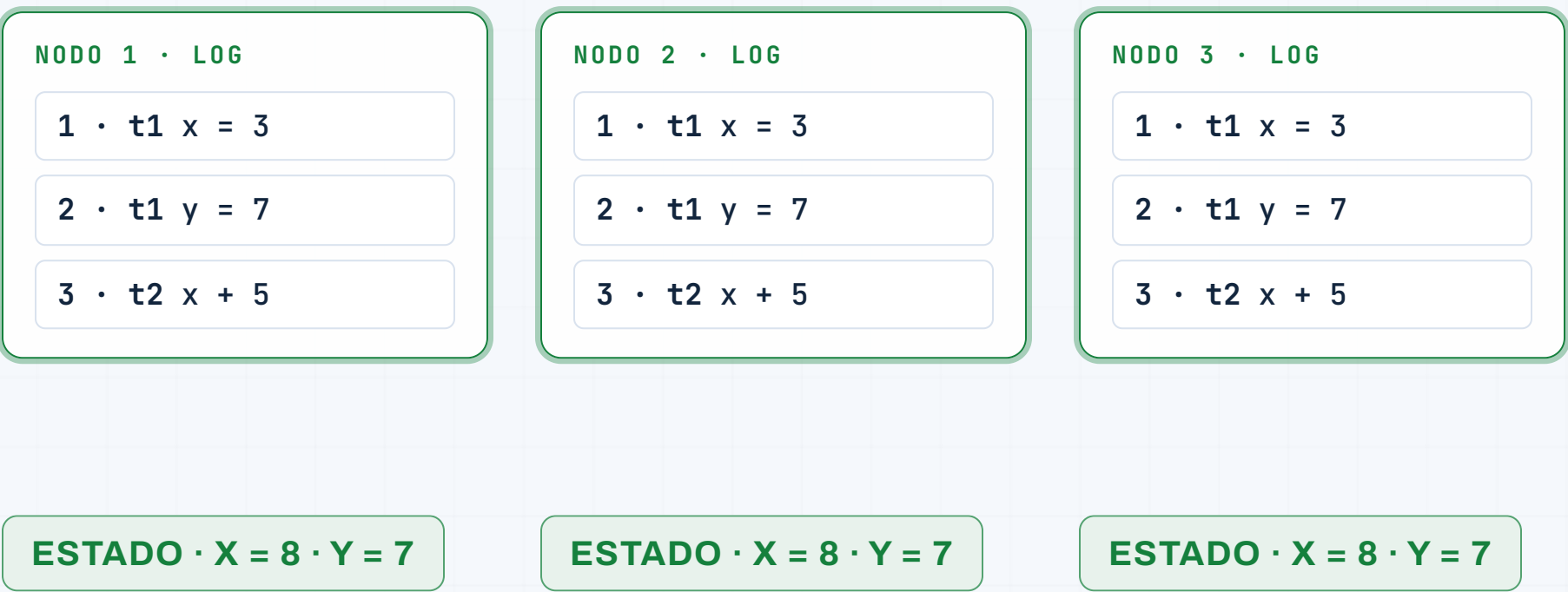
MÁQUINA DE ESTADOS

Todos aplican **las mismas entradas en el mismo orden** con lógica determinista, y todos llegan al mismo estado.

POR QUÉ ASÍ

- Se replica el **qué** la **operación**, no el resultado
- Mismo orden** todos aplican lo mismo en la misma secuencia
- Cada **entrada** lleva su índice y su término

TRES NODOS · LA MISMA LISTA, EN EL MISMO ORDEN



MISMAS OPERACIONES Y MISMO ORDEN · MISMO ESTADO



COMO SI FUERA

La libreta de movimientos de una tienda: no se copia el saldo final, se copian **los movimientos en orden**, y con esa lista cualquiera vuelve a sacar el saldo.

EN UNA FRASE Si todos aplican **las mismas operaciones en el mismo orden**, todos llegan al mismo estado.

Un voto por término.

TÉRMINO

Número que solo crece: **un líder por término**, y cada nodo vota a lo más una vez en cada término.

POR QUÉ NO HAY DOS LÍDERES

Dos mayorías de 5 **siempre comparten** a alguien
Ese alguien ya votó una vez en ese término
Empate timeout aleatorio de 150 a 300 ms y otra ronda



B Y D PIDEN VOTO EN EL TÉRMINO 5



COMO SI FUERA

Una asamblea donde cada socio **vota una vez por ronda** y la ronda tiene número: dos candidatos no pueden ganar la misma ronda.

EN UNA FRASE **Puede haber varios líderes en la historia, pero nunca dos en el mismo término.**

Mayoría

para las dos cosas.

DOS VOTACIONES

Elegir líder y comprometer cada entrada: las dos se deciden por mayoría del clúster completo.

CON 5 SERVIDORES

Comprometida la entrada está en el log de **3 de 5**
Entonces el líder la aplica y responde OK
Se traslapan quien guardó la entrada votó en la elección

Servidor 1
líder · 3 de 5 votos

ELEGIR · 3 DE 5

Servidor 2

log · 6

Servidor 3

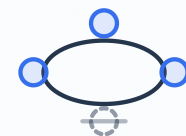
log · 6

Servidor 4

log · 6 · lento

Servidor 5

caído



COMO SI FUERA

El mismo quórum que eligió al presidente de la mesa **tiene que firmar cada acta**: sin tres firmas de cinco, el acta no vale.

EN UNA FRASE

Una entrada no está segura cuando el líder la escribe: **está segura cuando la tiene la mayoría.**

¿Cuándo está

comprometida?

COMPROBACIÓN 3

Raft, 5 servidores. El líder manda una entrada al log. ¿Cuándo la considera **comprometida**?

ANTES DE RESPONDER

Escrita el líder ya la puso en su propio log
Comprometida ya **no se puede perder**
Elige una A, B, C o D, y di por qué



- A · EN SU PROPIO LOG
- B · MAYORÍA, 3 DE 5
- C · LOS 5 LA REPLICARON
- D · EL LÍDER YA RESPONDIÓ



COMO SI FUERA

Un acuerdo que el secretario apuntó en su copia. ¿Ya es firme, o le falta algo? **La respuesta cambia si el secretario desaparece.**

EN UNA FRASE **No preguntes si está escrita: pregunta **qué sobrevive si el líder desaparece.****

Quién gana y quién repara.

LOG AL DÍA

Solo gana la elección **quien ya tiene todo lo comprometido**; el rezagado se alinea con el log del líder.

CÓMO SE CUIDA LO COMPROMETIDO

Al votar nadie vota por un log **más atrasado** que el suyo

Al volver el líder retrocede hasta donde coinciden

De ahí reenvía todo en orden; lo que sobra se borra

CADA ENTRADA · ÍNDICE · TÉRMINO · OPERACIÓN

LÍDER · LOG COMPLETO

1 · t1 x = 3

2 · t1 y = 7

3 · t2 x + 5

4 · t2 x - 2

REZAGADO · VUELVE

1 · t1 x = 3

2 · t1 y = 7

falta 3 estuvo caído

falta 4 estuvo caído

SE POSTULA · VOTO NEGADO: TU LOG VA ATRÁS



COMO SI FUERA

El compañero que faltó no vuelve a copiar el cuaderno entero: busca **la última clase que sí tiene** y copia de ahí para adelante.

EN UNA FRASE El log del líder manda: **el que volvió se alinea, no negocia.**

Lo que cuesta.

TOLERANCIA

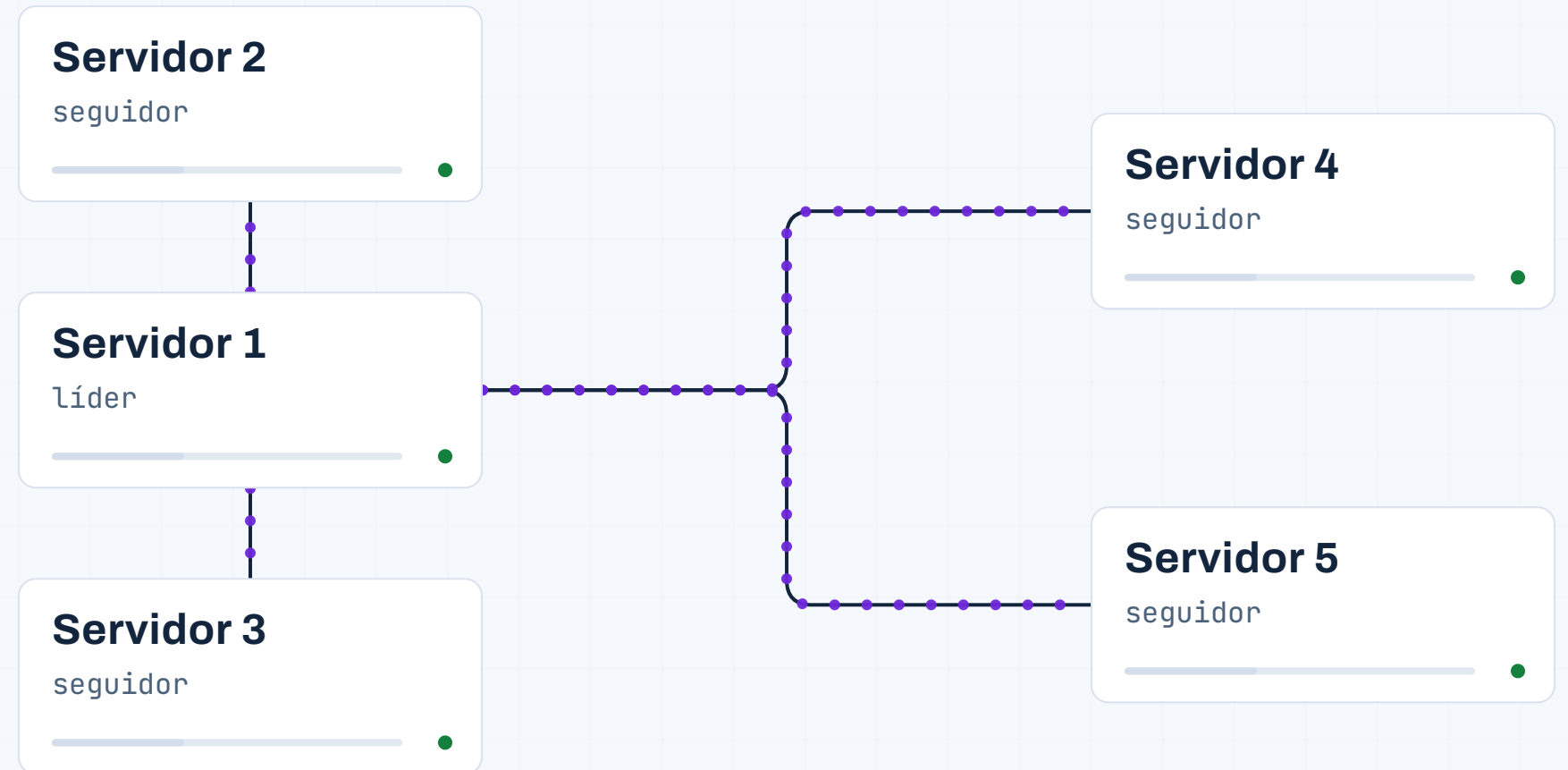
Tolerar f caídas pide **$2f + 1$ nodos**, y cuando se parte la red, la minoría no puede decidir nada.

LA CUENTA

3 nodos toleran 1 · **4 toleran lo mismo**

5 nodos toleran 2

Más nodos cada escritura **espera a más**



COMO SI FUERA

Dos salones separados por una puerta cerrada: **el que tiene más de la mitad de la asamblea sesiona**; el otro espera aunque todos estén presentes.

EN UNA FRASE

La mayoría de Raft es la decisión de CAP tomada de antemano: **prefiere no contestar a contestar mal.**

Pocos nodos que deciden.

COORDINACIÓN

Servicio que guarda **datos chicos que cambian lento** y los replica con consenso en 3 o 5 nodos.

ETCD, ZOOKEEPER, CONSUL

Tamaño impar, **no más de 7**; 5 toleran 2

Ofrecen locks, fencing, latido y avisos (watch)

Eureka prefiere A: con menos del 85 % de latidos en 15 min **no da de baja a nadie**

kubectl

apply · get pods

kube-apiserver

la única puerta

etcd 1

seguidor

etcd 2

líder · atiende consenso

etcd 3

seguidor

3 MIEMBROS · RAFT ·
KUBERNETES



COMO SI FUERA

La ventanilla contesta y sonríe, pero **lo que vale es el libro de atrás**: si ahí no quedó escrito, para la escuela nunca pasó.

EN UNA FRASE

El consenso se subcontrata: **miles de servidores le preguntan a 3 o 5.**

Lo que falta

en Compose.

EL LÍMITE DE COMPOSE

Compose **no revive, no reparte y vive en un solo host**: producción necesita a alguien que piense por ti.

TRES FALTAS

Revivir restart actúa si el proceso **sale**, no si está unhealthy

Repartir --scale da copias, no balanceo

Un host si cae la máquina, caen todas

Si transferencia muere

tú la levantas

Si hay más tráfico

tú escalas

Si el healthcheck falla

nadie la reinicia

transferencia

healthy

Docker

restart: always

COMO SI FUERA

Una orquesta sin director: cada músico toca bien, pero **si uno se calla, nadie lo nota ni lo reemplaza.**

EN UNA FRASE **Compose es perfecto para desarrollo; producción necesita algo que vigile y corrija solo.**

Varias máquinas.

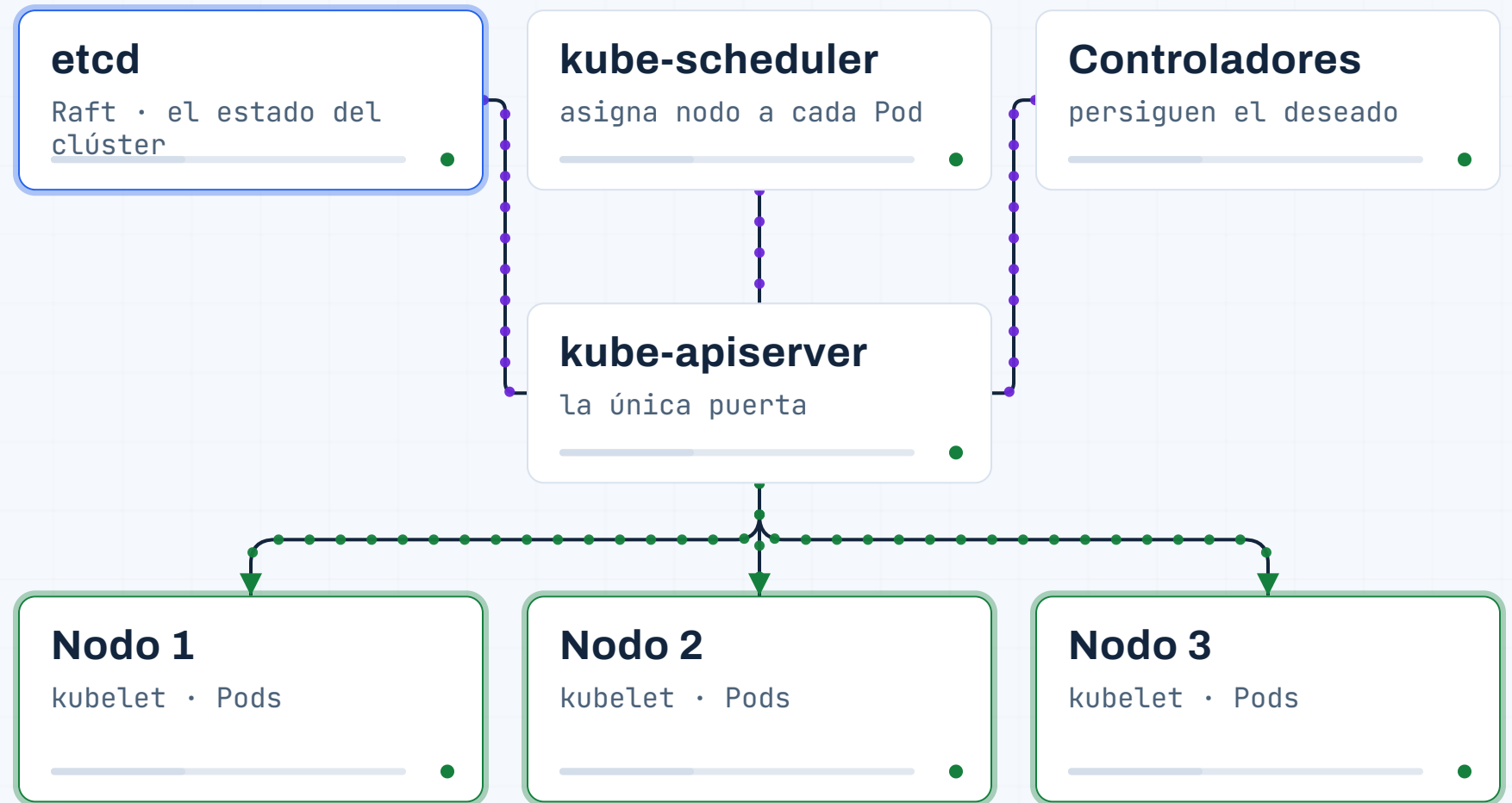
CLÚSTER

Un **plano de control** más un conjunto de nodos que corren las aplicaciones en contenedores.

QUIÉN HACE QUÉ

Nodos hospedan los Pods; kubelet en cada uno
Plano de control gobierna nodos y Pods
Producción el plano vive en **varias máquinas**

PLANO DE CONTROL



COMO SI FUERA

Una escuela con muchos salones y una sola dirección: **si un salón se inunda, la clase sigue en otro**, porque quien asigna salones no está adentro.

Di qué quieres, no cómo.

ESTADO DESEADO

Describes lo que quieres y **Kubernetes cambia el estado real hasta alcanzarlo**, a un ritmo controlado.

CÓMO SE PIDE

Sí tres copias de **cuenta:1.4** y una dirección estable

No arranca esto en esta máquina
kube-scheduler elige el nodo de cada Pod

Estado deseado

replicas: 3 · cuenta:1.4

compara

CUENTA

deseado

3

real

2 → 3

ajusta

HAY 2, QUIERO 3: CREA 1

cuenta-6b7a9-7f2a

nodo 1 · Running

cuenta-6b7a9-9c4d

nodo 2 · Running

cuenta-6b7a9-b81e

nodo 1 · nuevo



COMO SI FUERA

Le dices a la app del taxi **a dónde quieres llegar**, no por cuáles calles: si una se cierra, recalcula sola.

EN UNA FRASE

Mientras el estado deseado no cambie, **el clúster lo persigue aunque nadie esté viendo.**

El termostato

que nunca duerme.

CONTROLADOR

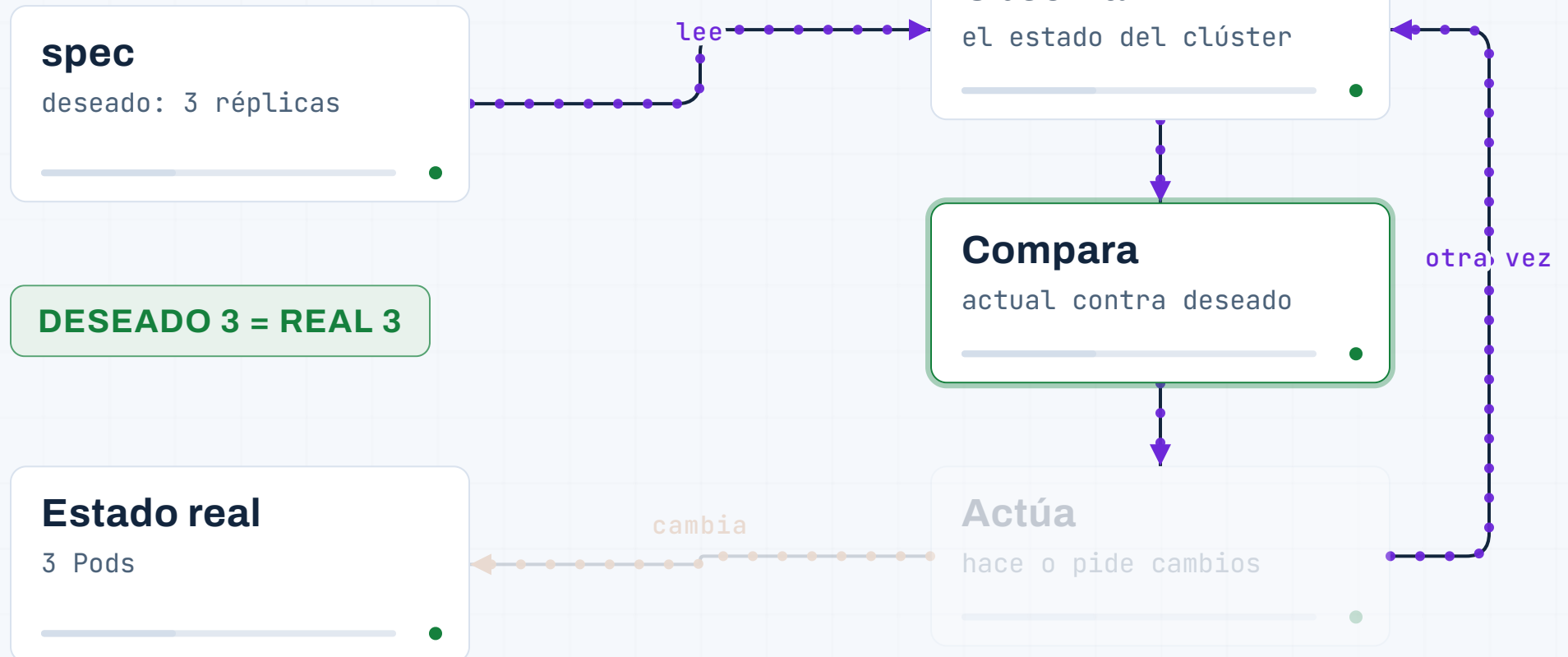
Bucle que observa el estado del clúster y **hace o pide cambios** para acercar el actual al deseado.

EL BUCLE

Observa el estado del clúster

Compara actual contra deseado

Actúa hace o pide cambios, y **vuelve a empezar**



COMO SI FUERA

Pones la temperatura que quieres y te vas: **el termostato mide y corrige toda la noche**, sin que nadie se lo recuerde.

La unidad mínima.

POD

La unidad de cómputo más pequeña:
uno o más contenedores que comparten red y almacenamiento.

LO QUE HAY QUE SABER

Una IP por Pod; sus contenedores la comparten
Efímero desechable; rara vez se crea a mano
Lo crea un Deployment, no una persona

Pod

cuenta-6b7a9-k2xq

Contenedor

java · cuenta:1.4

IP 10.244.1.7

Pod

cuenta-6b7a9-m8rt · dos contenedores

Contenedor

java · cuenta:1.4

Contenedor

agente de logs

IP 10.244.1.9 · UNA SOLA



COMO SI FUERA

Dos escritorios en el mismo cubículo: **una sola extensión y un solo archivero** para los dos, y si se desarma el cubículo se van juntos.

EN UNA FRASE

Kubernetes no programa contenedores: **programa Pods**, y el contenedor va adentro.

Quien cuenta las copias.

DEPLOYMENT

Gestiona Pods sin estado; **debajo, el ReplicaSet mantiene estable el número de Pods** creando y borrando.

EN EL EJEMPLO

replicas: 3 el número deseado va en `.spec.replicas`
template la plantilla con la que se crea cada Pod
ReplicaSet crea y borra hasta llegar al número

Deployment

cuenta · replicas: 3

ReplicaSet

cuenta-6b7a9

REPLICASET DESEADO 3 · REAL 3

Pod

cuenta-6b7a9-k2xq

Pod

cuenta-6b7a9-m8rt

Pod

cuenta-6b7a9-p7vd



COMO SI FUERA

El contrato del turno pide **tres personas en el piso**, no dice sus nombres: si una falta, entra otra y el número vuelve a tres.

Nunca vuelve el mismo.

REEMPLAZO DE UN POD

Se programa una sola vez; **nunca se reprograma a otro nodo: se reemplaza por un Pod nuevo casi idéntico** con otro nombre y otra IP.

CUANDO MUERE EL NODO

Sin latido el kubelet deja de renovar su Lease y el nodo queda NotReady
El conteo el ReplicaSet ve dos y quiere tres
El nuevo otro nodo, otro nombre, otra IP

Nodo A

kubelet · Lease renovado

Pod

cuenta-6b7a9-k2xq ·
10.244.1.7

Nodo B

kubelet · Lease renovado

Pod

cuenta-6b7a9-m8rt

Pod

cuenta-6b7a9-p7vd

CRASHLOOPBACKOFF **SUBE RESTARTS**



COMO SI FUERA

Si se cae el piso entero, nadie repara ese cubículo: **arman otro en otro piso, con otra extensión**, y el trabajo sigue en el nuevo.

EN UNA FRASE **No esperes a que vuelva el mismo Pod: **vuelve otro**, y por eso adentro no se le guarda nada.**

Una puerta estable.

SERVICE

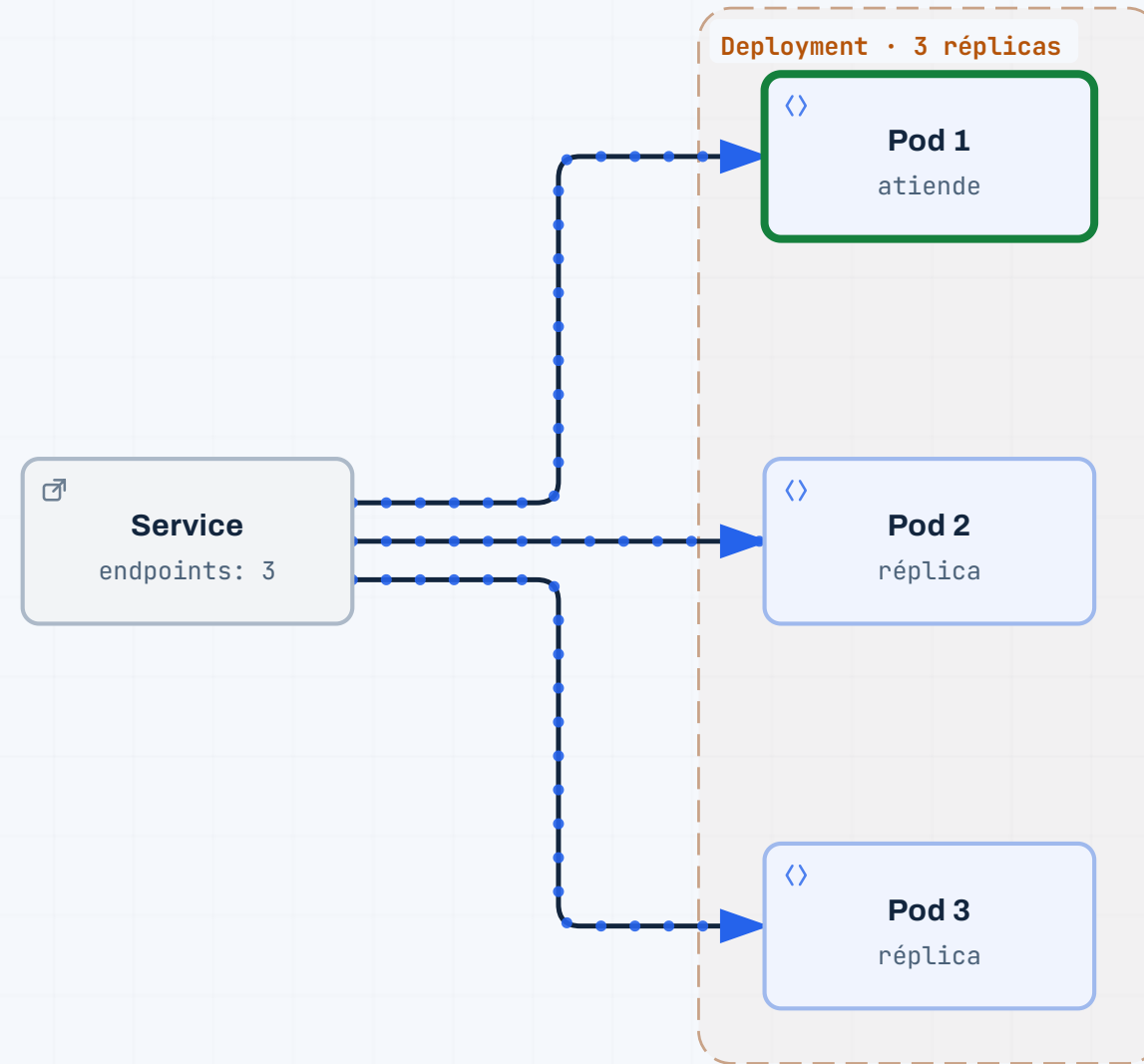
Expone una aplicación que corre en uno o más Pods: **los encuentra por etiquetas y mantiene la lista al día.**

LO QUE DA

Nombre DNS estable: cuenta, aunque las IPs cambien

Endpoints la lista de Pods vivos, por etiqueta

kube-proxy en cada nodo, lleva el tráfico al Pod de esa lista



COMO SI FUERA

Una extensión que no cambia aunque **roten las personas que contestan**: kube-proxy es quien pasa cada llamada al escritorio que sí está ocupado.

Vivo, listo y el acuerdo.

SONDAS

Liveness **reinicia** el contenedor que no responde; readiness **lo saca del Service** hasta que esté listo.

LO QUE CAMBIA

Docker no reinicia por unhealthy; **Kubernetes** sí **Spring Boot** /actuator/health/liveness y /readiness **etcd** sin mayoría no hay cambios, **los Pods siguen**



EN UNA FRASE

El plano de control prefiere no decidir antes que decidir mal: **es CP**, y tus Pods siguen atendiendo.

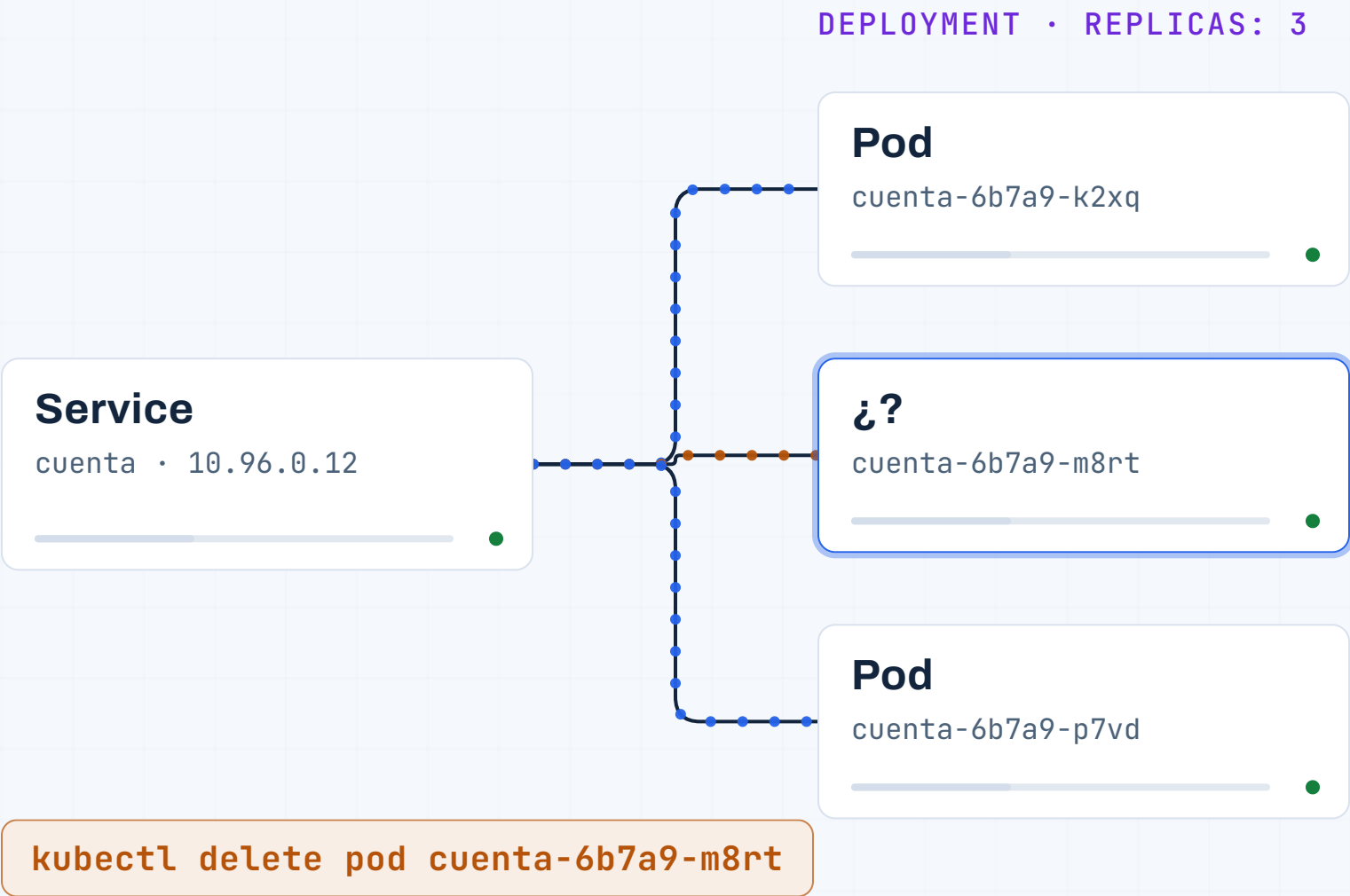
¿Qué predices?

COMPROBACIÓN 4

Un Deployment cuenta con 3 réplicas; corres `kubectl delete pod` sobre uno de sus Pods. ¿Qué predices?

OPCIONES

- A el mismo Pod, con el mismo nombre
- B un Pod nuevo, otro nombre, AGE 0
- C restaura el Pod de un respaldo



EN UNA FRASE Comprométete con una opción antes de ver el resultado: **predecir primero es lo que hace que se quede.**

Réplicas o máquina más grande.

ESCALAR

Cambiar el **número de réplicas** (horizontal) o los **recursos de cada una** (vertical).

QUÉ EXIGE CADA UNA

Horizontal procesos **sin estado** y algo que reparta
Vertical un solo punto de falla; no cambia código
etcd cinco con **discos rápidos**, no diez lentos



cuenta

app · sin estado

A · HORIZONTAL

réplica 1

mismo tamaño

réplica 2

mismo tamaño

réplica 3

mismo tamaño

B · VERTICAL

1 instancia

más CPU y RAM



COMO SI FUERA

Un tráiler de treinta toneladas se detiene con **una llanta pinchada**; treinta camionetas siguen andando si una choca, mientras ningún chofer se sepa la ruta de memoria.

La decisión

de hoy.

LA DECISIÓN

Buen Fin: Mexi Banco recibe **el triple de carga de golpe**. ¿Tres réplicas del mismo tamaño o una máquina tres veces más grande?

EN TRES LÍNEAS

Elección una de las dos

Porque la razón, con el requisito enfrente

Sacrificio lo que pierdo al elegir

Mexi Banco

Buen Fin · carga x 3

ELECCIÓN

PORQUE

SACRIFICIO

Réplicas

horizontal · n copias
iguales

Máquina más grande

vertical · 1 instancia



COMO SI FUERA

Firmar sin leer la letra chica: **la decisión se cumple igual**, y el costo que no dijiste aparece solo, más tarde y más caro.

EN UNA FRASE

Aquí no hay respuesta correcta: la respuesta completa es la elección, la razón y el sacrificio.

43 / DEMO

Un banco,

cinco servicios.

PARTIR

Poner **cada módulo en su propio proceso**, dueño de su base, y que se hablen por la red.

Mexi Banco v05.1

un deployable · 5 módulos · 1 base



V05.1 → V06A

Cinco servicios cuenta, movimiento, transferencia, notificación, spei

Entre ellos HTTP, con la URL por nombre de Compose

Datos una base por servicio



COMO SI FUERA

Una oficina con cinco escritorios que se vuelve cinco oficinas: **ya no basta con voltear, hay que llamar.**

EN UNA FRASE

Cada llamada que antes era un método ahora **cruza la red**, con todo lo que vimos desde la S01.

44 / DEMO

Un comando, seis contenedores.

DOCKER COMPOSE UP

Un archivo levanta **todo separado y conectado**, y espera a que cada pieza esté sana.

LO QUE VES

Puertos 8081 a 8085, uno por servicio
docker compose ps los seis en **healthy**
Nombres nadie escribe una IP

```
docker compose up --build -d
```

cuenta

8081 · starting

movimiento

8082 · starting

transferencia

8083 · starting

notificacion

8084 · starting

spei

8085 · starting

db

postgres · healthy



PLAN B

Si la demo falla en vivo, esta lámina **es la demo**: el mismo estado que debería verse en la terminal.

EN UNA FRASE

Compose ya sabía levantar dos contenedores; **ahora levanta un sistema.**

45 / DEMO

Se rompió

al partir.

TRANSFERENCIA PARTIDA

Cargar y abonar ya son dos llamadas por red: **la transferencia dejó de ser una transacción.**

LO QUE SE ROMPIÓ

Cinco puertos el cliente tiene que saberlos todos
Cargar sí, abonar no el dinero queda en el aire
Es commit atómico o sea, consenso; sagas en la S12



COMO SI FUERA

Mandar el dinero en dos sobres por correo: **si se pierde el segundo, el primero ya salió.**

EN UNA FRASE

Partir quita el acoplamiento y también la transacción: lo que era gratis ahora se diseña.

Para la S07.

S07

Taller del Arquitecto: **el caso de tu equipo**, con la primera decisión registrada.

FECHAS

Reporte v05.1 vence el 2026-09-27

S07 2026-10-03, sin lab: todo es taller

Tarea: clona v06a y levántalo

`docker compose up --build · los seis en healthy`

Pregunta 1

¿qué URL usa transferencia para hablar con cuenta, y dónde está escrita?

Pregunta 2

¿qué pasa con el dinero si detienes cuenta a media transferencia?

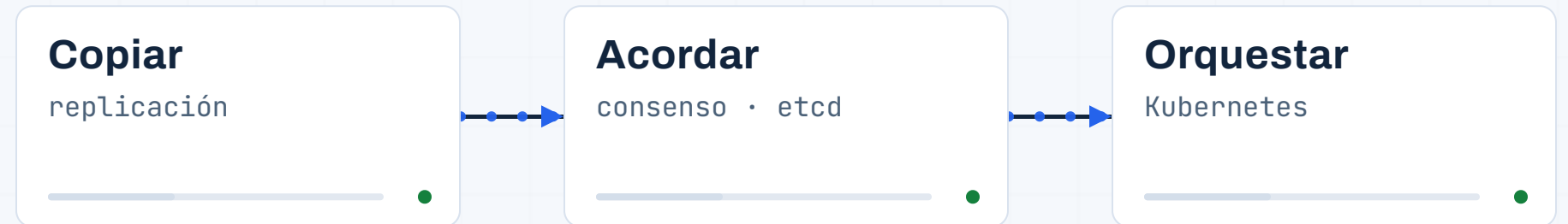
EN UNA FRASE

Trae la idea del caso de tu equipo: **en el taller se decide sobre lo tuyo.**

Un principio.

EL PRINCIPIO

Copiar da disponibilidad; ponerse de acuerdo da consistencia; y **Kubernetes usa el acuerdo para mantener vivas tus copias.**



Y CUANDO SE CORTA LA RED: C O A

48 / CIERRE

Ticket

de salida.

TICKET DE SALIDA

¿Qué operación de tu proyecto tiene que ser consistente aunque se caiga la red, y cuál puede ser eventual?

FUENTES DE LA SESIÓN

Kleppmann y Riccomini Designing Data-Intensive Applications, 2.^a ed., caps. 6 y 10

Brewer CAP Twelve Years Later, IEEE Computer, 2012 (Lectura 5)

Abadi Consistency Tradeoffs in Modern Distributed Database System Design, 2012

Ongaro y Ousterhout In Search of an Understandable Consensus Algorithm, 2014

Documentación etcd (FAQ) y Kubernetes (arquitectura, controladores, sondas)

EN UNA FRASE

Una línea en el chat antes de irte: **una operación consistente y una eventual.**